

- [TPM-backed LUKS decryption](#) (page 179) to automate decryption of disks

User management

User management is a critical part of maintaining a secure system. Ineffective user and privilege management often leads to a system being compromised. Therefore, it is important that you understand how to protect your server through simple and effective user account management techniques.

Where is root?

Ubuntu developers decided to disable the administrative root account by default in all Ubuntu installations. This does not mean that the root account has been deleted, or that it may not be accessed. Instead, it has been given a password hash that matches no possible value, and so may not log in directly by itself.

Instead, the `sudo` utility (“superuser do”) is used to carry out system administrative duties. `sudo` allows an authorized user to temporarily elevate their privileges using their own password instead of having to know the password belonging to the root account. This provides accountability for all user actions, and gives the administrator control over which actions a user can perform with said privileges.

`sudo-rs`

From Ubuntu 25.10 (Questing Quokka) onward, `sudo` is provided by the `sudo-rs` package (a rust implementation). While `sudo-rs` is not 100% compatible with `sudo.ws`, the majority of common use cases are supported and the change should be invisible to most users.

A list of key differences is [maintained as a reference](#) (page 854)

The original `sudo` utility (maintained by Todd C. Miller), `sudo.ws` remains supported in the 25.10 and the subsequent 26.04 LTS. You will find these utilities installed with `.ws` suffix (for example `sudo.ws`, `visudo.ws`, etc.). Switching the default back to `sudo.ws` is not recommended, but if you need to do so, follow these steps:

Interactive:

```
$ update-alternatives --config sudo
```

Non-interactive:

```
$ update-alternatives --set sudo /usr/bin/sudo.ws
```

You can always switch back to `sudo-rs` using:

```
$ update-alternatives --set sudo /usr/lib/cargo/bin/sudo
```

You can learn more about the motivation for this change in the blog post [Adopting sudo-rs By Default in Ubuntu 25.10](#)⁸⁶

⁸⁶ <https://discourse.ubuntu.com/t/adopting-sudo-rs-by-default-in-ubuntu-25-10/60583>

The rest of the article should work the same with both `sudo.ws` and `sudo-rs`.

Enabling the root account

If for some reason you wish to enable the root account, you will need to give it a password:

```
$ sudo passwd
```

`sudo` will prompt you for your password, and then ask you to supply a new password for root as shown below:

```
[sudo: authenticate] Password: (enter your own password)
New password: (enter a new password for root)
Retype new password: (repeat new password for root)
passwd: password updated successfully
```

Disabling the root account password

To disable the root account password, use the following `passwd` syntax:

```
$ sudo passwd -l root
```

You can learn more about `sudo` by reading the [sudo\(8\)](https://manpages.ubuntu.com/manpages/resolute/man8/sudo.8.html)⁸⁷ manual page: `man sudo`

By default, the initial user created by the Ubuntu installer is a member of the group `sudo` which is added to the file `/etc/sudoers` as an authorized `sudo` user. To give any other account full root access through `sudo`, add them to the `sudo` group.

Listing, adding, and deleting local users

Managing local users and groups differs very little from most other [GNU/Linux](#) operating systems. Ubuntu and other Debian-based distributions encourage the use of the `adduser` package for account management.

Local users and groups are those defined in the `/etc/passwd` and `/etc/group` files, respectively. Via the Name Service Switch ([NSS](#)) mechanism, however, user and group accounts can also come from other sources, like remote Active Directory or Samba servers, OpenLDAP directory services, and more. Managing accounts in those services is an entirely different matter, and not covered in this document.

⁸⁷ <https://manpages.ubuntu.com/manpages/resolute/man8/sudo.8.html>

Listing local users

The local users are defined one per line in the `/etc/passwd` text file. The `cat /etc/passwd` command can be used to list its contents (only a few lines shown below, for brevity):

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
...
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
```

Tip:

The structure of the `/etc/passwd` file is explained in the `passwd(5)`⁸⁸ manual page.

⁸⁸ <https://manpages.ubuntu.com/manpages/resolute/man5/passwd.5.html>

There are basically three types of local users:

- system pre-installed users: These are system users essential to a working Linux system, like `root`, `man`, `lp`, and others. Their `UIDs` are in the range between 0 and 99.
- system users: These users are created dynamically, as services are installed. The UID range here is between 100 and 999.
- Regular users: Finally, the remaining range is usable for “real” users, generally representing persons using the system. This UID range starts at 1000 and, by convention, ends at 59999.

Tip:

For more details about these UID ranges, including groups and other configuration options for adding users, check the `adduser.conf.5`⁸⁹ manual page.

⁸⁹ <https://manpages.ubuntu.com/manpages/resolute/man5/adduser.conf.5.html>

To list all local usernames, we can list the contents of `/etc/passwd` with some string manipulation. For example:

```
$ cat /etc/passwd | cut -d : -f 1
```

This command will cut each line at the first column, using the “:” character as the delimiter, and result in an output showing all local users:

```
root
daemon
bin
sys
...
```

If you want to only show regular users, that is, users with an UID greater than 1000, we can use a regular expression for this filtering:

```
$ cat /etc/passwd | cut -d : -f 1,3 | grep -E ':[0-9]{4,}'
```

The regular expression `:[0-9]{4,}` means all digits, repeated 4 times or more. The output will be similar to this:

```
nobody:65534
ubuntu:1000
```

Tip:

By convention, the `nobody` users has an UID of 65534. That originated back when the user range was limited to 16 bits (so, 0-65535), and is kept for compatibility reasons.

A note about remote users

There is another command that, at first glance, looks like it lists all local users as well: `getent passwd`. What this tool is doing, however, is listing all entries from the `passwd` database, and this will include any NSS modules that might be installed and configured.

On a default installation of Ubuntu, with no extra configuration, that will indeed list only the local users, but not in the general case. If you have, for example, `libnss-ldap` installed and configured, the output of `getent passwd` will list all local users, **plus all the remote users** it finds in the LDAP server that it was configured with.

Performing such an exhaustive enumeration is frowned upon, because that can incur a large amount of network traffic, and place an unreasonable burden on the remote server. In fact, most NSS services block such enumeration actions by default.

For more information on the Name Service Switch (NSS) mechanism, please consult the [nss\(5\)](https://manpages.ubuntu.com/manpages/resolute/man5/nss.5.html)⁹⁰ and [nsswitch.conf\(5\)](https://manpages.ubuntu.com/manpages/resolute/man5/nsswitch.conf.5.html)⁹¹ manual pages.

Add a user

To add a user account, use the following syntax, and follow the prompts to give the account a password and identifiable characteristics, such as a full name, phone number, etc:

```
$ sudo adduser username
```

⁹⁰ <https://manpages.ubuntu.com/manpages/resolute/man5/nss.5.html>

⁹¹ <https://manpages.ubuntu.com/manpages/resolute/man5/nsswitch.conf.5.html>

Delete a user

To delete a user account and its primary group, use the following syntax:

```
$ sudo deluser username
```

Deleting an account does not remove their respective home folder. You must decide whether or not to delete the folder manually, or to keep it according to your desired retention policies.

Remember, any user added later with the same UID/GID as the previous owner will now have access to this folder if you have not taken the necessary precautions.

You may want to change the UID/GID values to something more appropriate, such as the root account, and perhaps even relocate the folder to avoid future conflicts:

```
$ sudo chown -R root:root /home/username/
```

```
$ sudo mkdir /home/archived_users/
```

```
$ sudo mv /home/username /home/archived_users/
```

Lock or unlock a password

To temporarily lock a user password, use the following syntax:

```
$ sudo passwd -l username
```

Similarly, to unlock a user password:

```
$ sudo passwd -u username
```

Add or delete a group

To add or delete a personalised group, use the following syntax, respectively:

```
$ sudo addgroup groupname
```

```
$ sudo delgroup groupname
```

Add a user to a group

To add a user to a group, use the following syntax:

```
$ sudo adduser username groupname
```

User profile security

When a new user is created, the `adduser` utility creates a brand new home directory named `/home/username`. The default profile is modelled after the contents found in the directory of `/etc/skel`, which includes all profile basics.

If your server will be home to multiple users, you should pay close attention to the user home directory permissions to ensure confidentiality.

Ubuntu 21.10 Impish and later

Home directories are created with private permissions (`0750` or `drwxr-x---`) by default. This ensures that only the user and members of their group can access the directory.

Ubuntu 21.04 Hirsute and earlier

Home directories were created with world read/execute permissions (`0755` or `drwxr-xr-x`) by default. This meant that all users on the system could browse and access the contents of other users home directories, which may not be suitable for your environment.

To verify your current user home directory permissions, use the following syntax:

```
$ ls -ld /home/username
drwxr-xr-x  2 username username    4096 2007-10-02 20:03 username
```

The output shows that the directory `/home/username` has world-readable permissions. You can remove the world readable-permissions using the following command:

```
$ sudo chmod 0750 /home/username
```

Note:

Some people use the recursive option (`-R`) indiscriminately, which modifies all child folders and files. However, this is not necessary and may have undesirable/unintended consequences. Modifying only the parent directory is enough to prevent unauthorised access to anything below the parent.

A more efficient approach would be to modify the `adduser` global default permissions when creating user home folders. To do this, edit the `/etc/adduser.conf` file and modify the `DIR_MODE` variable to something appropriate, so that all new home directories will receive the correct permissions.

```
DIR_MODE=0750
```

After correcting the directory permissions using any of the previously mentioned techniques, verify the results as follows:

```
$ ls -ld /home/username
drwxr-x--- 2 username username 4096 2007-10-02 20:03 username
```

Password policy

A strong password policy is one of the most important aspects of your security posture. Many successful security breaches involve simple **brute force** and **dictionary** attacks against weak passwords.

If you intend to offer any form of remote access involving your local password system, make sure you address minimum password complexity requirements, maximum password lifetimes, and frequent audits of your authentication systems.

Minimum password length

By default, Ubuntu requires a minimum password length of 6 characters, as well as some basic entropy checks. These values are controlled in the file `/etc/pam.d/common-password`, which is outlined below.

```
password [success=1 default=ignore] pam_unix.so obscure sha512
```

To adjust the minimum length to 8 characters, change the appropriate variable to `minlen=8`. The modification is outlined below:

```
password [success=1 default=ignore] pam_unix.so obscure sha512 minlen=8
```

Note:

Basic password entropy checks and minimum length rules do not apply to the administrator using `sudo`-level commands to setup a new user.

Password expiration

When creating user accounts, you should make it a policy to have a minimum and maximum password age, forcing users to change their passwords when they expire.

To view the current status of a user account:

```
$ sudo chage -l username
```

The output below shows interesting facts about the user account, namely that there are no policies applied:

```
Last password change           : Jan 20, 2015
Password expires               : never
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

To set any of these values, use the `chage` ("change age") command, and follow the interactive prompts:

```
$ sudo chage username
```

The following is also an example of how you can manually change the explicit expiration date (-E) to 01/31/2015, minimum password age (-m) of 5 days, maximum password age (-M) of 90 days, inactivity period (-I) of 30 days after password expiration, and a warning time period (-W) of 14 days before password expiration:

```
$ sudo chage -E 01/31/2015 -m 5 -M 90 -I 30 -W 14 username
```

To verify changes, use the same syntax as mentioned previously:

```
$ sudo chage -l username
```

The output below shows the new policies that have been established for the account:

```
Last password change           : Jan 20, 2015
Password expires               : Apr 19, 2015
Password inactive              : May 19, 2015
Account expires                : Jan 31, 2015
Minimum number of days between password change : 5
Maximum number of days between password change : 90
Number of days of warning before password expires : 14
```

Other security considerations

Many applications use alternate authentication mechanisms that can be easily overlooked by even experienced system administrators. Therefore, it is important to understand and control how users authenticate and gain access to services and applications on your server.

SSH access by disabled passwords

Disabling or locking a user password will not prevent a user from logging into your server remotely if they have previously set up SSH public key authentication. They will still be able to gain shell access to the server, without the need for any password. Remember to check the user's home directory for files that will allow for this type of authenticated SSH access, e.g. `/home/username/.ssh/authorized_keys`.

Remove or rename the directory `.ssh/` in the user's home folder to prevent further SSH authentication access.

Be sure to check for any established SSH connections by the disabled account, as it is possible they may have existing inbound or outbound connections – then `kill` any that are found.

```
$ w | grep username (to get the pts/# terminal)
```

```
$ sudo kill -f pts/#
```

Restrict SSH access to only user accounts that should have it. For example, you may create a group called `sshlogin` and add the group name as the value associated with the `AllowGroups` variable located in the file `/etc/ssh/sshd_config`:

```
AllowGroups sshlogin
```

Then add your permitted SSH users to the group `sshlogin`, and restart the SSH service.

```
$ sudo adduser username sshlogin
```

```
$ sudo systemctl restart ssh.service
```

External user database authentication

Most enterprise networks require centralised authentication and access controls for all system resources. If you have configured your server to authenticate users against external databases, be sure to disable the user accounts both externally and locally. This way you ensure that local *fallbacks* authentication is not possible.